RUBINA DANGOL MAHARJAN
AI FELLOWSHIP
FUSEMACHINE

4th May 2026

WeeK-1 Assignment

Data Wrangling: Data Scientist at the regional health center

Presented to
Pralisha Kansakar

Presented by
Rubina Maharjan

TABLE OF CONTENTS

ABOUT

Missing Value Assessment and Handling in the Heart Attack Data Wrangling Notebook

This notebook focuses on data wrangling for a heart attack risk dataset divided into three separate files:

1. **Patient demographics**
2. **Clinical data**
3. **Lifestyle factors**

The main objective of the wrangling process is to inspect data quality, identify missing values, clean formats, and prepare the datasets for later exploratory analysis and modeling.

A major part of the notebook is dedicated to checking whether any values are missing and deciding whether they should be dropped, filled, or handled using a column-specific strategy.

Missing Value Detection

Checks missing value separately for each datasets.

code:

```
demographics.isnull().sum()
clinical.isna().sum()
lifestyle.isna().sum()
```

Both .isnull() and .isna() perform the same function in pandas: they detect missing values such as NaN, None, or blank values interpreted as null.

Dataset	Columns Checked	Missing Value Found
Demographics	Patient ID, Age, Sex, Income, Country, Continent, Hemisphere	0 in all columns
Clinical	Patient ID, Cholesterol, Blood Pressure, Heart Rate, Diabetes, Family History, Triglycerides, BMI	0 in all columns
Lifestyle	Patient ID, Smoking, Obesity, Alcohol Consumption, Exercise Hours Per Week, Diet, Stress Level, Heart Attack Risk	0 in all columns
The key finding is that **no missing values were found in any of the three datasets**.		

How Missing Values Were Handled

Since the missing value scan showed **zero missing values**, the notebook correctly avoids unnecessary imputation or row deletion.

Filling values:

`fillna()`	No values were filled.

Drop:

`dropna()`	No rows or columns were removed.

This is an appropriate decision because there was no actual missing data to replace or remove.

Instead of forcing an imputation strategy, this notebook keeps the original records and focuses on preparing the data structure for analysis.

Why No Imputation Was Used

Method	Meaning	When It Would Be Used
dropna()	Removes rows or columns with missing values	Useful when missing data is very small and random
fillna()	Replaces missing values with mean, median, mode, or another value	Useful when keeping dataset size is important
Column-specific strategy	Uses different rules for different variables	Useful when clinical and lifestyle data have different importance

However, none of these techniques were applied because the data did not require them.

This is a good data-cleaning decision because imputing values when nothing is missing would unnecessarily modify the dataset. In health-related data, this is especially important because clinical variables such as cholesterol, BMI, blood pressure, heart rate, and triglycerides represent real patient measurements. Changing them without need could distort later analysis.

Missing Value Treatment and Data Standardization

Summary

DATA CLEANING STEP	TECHNIQUE USED	REASON FOR USING IT
Column name cleaning	Removed spaces, converted names to lowercase, and replaced spaces with underscores	To make column names consistent and easier to use in Python
Text column cleaning	Used .astype(str).str.strip()	To remove extra spaces from categorical/text values
Numeric conversion	Used pd.to_numeric(..., errors="coerce")	To convert valid numbers properly and turn invalid values into NaN
Nullable integer conversion	Used .astype("Int64")	To allow integer columns to store missing values safely if any appear
Blood pressure cleaning	Split blood_pressure into systolic_bp and diastolic_bp	To convert combined blood pressure text into useful numerical features
Final missing value check	Used .isna().sum() after cleaning	To confirm whether any missing values appeared after datatype conversion
Missing value treatment decision	No mean, median, mode, or row deletion was applied	Because the original datasets did not contain direct missing values

The missing value treatment used in this notebook is best described as a **validation-based missing value handling approach**. The notebook did not fill or delete missing values because no direct missing values were found. Instead, it cleaned the data, corrected datatypes, exposed hidden invalid values as NaN, and then verified whether any missing values remained after cleaning.

Why the Chosen Strategy Was Appropriate

The chosen strategy can be described as a preserve-and-standardize approach.

Instead of dropping or filling values, the notebook:

1. Checked all datasets for missing values.
2. Confirmed that missing values were not present.
3. Preserved all original records.
4. Standardized column names.
5. Converted datatypes.
6. Split blood pressure into numerical systolic and diastolic values.
7. Used errors="coerce" as a safety check for invalid numeric data.

This was appropriate because the dataset was already complete. Applying unnecessary imputation would have added artificial assumptions. Dropping rows would have reduced the sample size without any benefit.

For a health-risk dataset, preserving real measurements is important because small changes in variables such as cholesterol, BMI, blood pressure, and triglycerides can affect risk interpretation.

Strengths of the Missing Value Handling

- Missing values were checked separately for each dataset.
- Clinical and lifestyle missingness were treated conceptually as different issues.
- No unnecessary imputation was performed.
- The notebook recognizes that missing data can be MCAR, MAR, or MNAR.
- Datatype conversion was done carefully.
- errors="coerce" was used to reveal hidden invalid values.
- The original records were preserved because the data was complete.

Possible Improvements

Improvement	Why It Helps
Check duplicate rows using .duplicated().sum()	Confirms that patient records are not repeated
Check missing values after datatype conversion	Ensures errors="coerce" did not create new NaN values
Check blank strings separately	Some missing values may appear as " " instead of NaN
Check unusual placeholders	Values like "Unknown", "N/A", "-", or "None" may not always be detected as missing
Show missing value percentages	Helps decide whether dropping or imputing would be acceptable
Visualize missingness with a heatmap	Useful if missing values exist in larger datasets

A useful additional check would be:

```
clinical_clean.isna().sum()
```

after all datatype conversions, because converting invalid strings to numbers can create new missing values.

Conclusion

The notebook correctly identifies that there are no missing values in the demographics, clinical, or lifestyle datasets. Because of this, the best handling strategy was to avoid both deletion and imputation.

The notebook's final approach is justified: it keeps all original records, avoids introducing artificial values, and focuses on cleaning structure and datatypes. This is especially suitable for health data, where unnecessary imputation could distort clinical meaning.

Overall, the missing value handling is appropriate because the dataset is complete, and the cleaning decisions support reliable downstream analysis.